Kidney-RAG

A teammate's guide to how we built a grounded, cited, safety-scored clinical assistant over four days.

Project	Kidney-RAG - CKD Clinical Decision Support (Lite)
Event	Creativa x Orange AI Hackathon, Aug 16-20 2026
Corpus	4 CKD guidelines - 666 indexed chunks
Stack	Python, MedEmbed, ChromaDB, BM25, FastAPI, Gemini/HF
Written for	Teammates who know Python but haven't built RAG before
Read in	~20 minutes

How to read this doc. Each day is one chapter. Chapters start with the goal in plain English, then the code we wrote, then what we measured. You can skim any chapter and still understand the next. The final chapter shows how everything fits together and how to run it.

What we built

Kidney-RAG is a clinical assistant that answers questions about chronic kidney disease (CKD) using only official guideline text - never its own medical opinion. Every answer includes a verbatim quote from the source, a citation with document name, section, and page number, and a confidence score. If the evidence in the corpus isn't strong enough to answer, the system refuses on purpose - a fluent wrong answer is more dangerous than 'I don't know'.

We indexed four public guidelines: KDIGO 2024 CKD (199 pages), KDIGO 2022 Diabetes in CKD (128 pages), NICE NG203 (78 pages), and USPSTF CKD Screening (6 pages). Together that's 666 searchable text chunks tagged with their exact source location. Every clinical answer must trace back to at least one of those chunks.

The four-day arc

Day	Goal	What shipped
1	Build the corpus and make it searchable	PDFs -> parsed JSON -> chunks -> embeddings -> ChromaDB index (666 chunks).
2	Make retrieval accurate. Measure it.	Hybrid semantic+BM25 search, 18-question eval set, Precision/Recall/Hit@k, chunk-siz
3	Add an LLM but keep it grounded	Recommendation/Excerpt/Citation format, retrieval-quality gate, refusal conditions, 3 LL
4	Prove it's safe. Give it a face.	Claim extractor, faithfulness + citation-accuracy metrics, 4-level uncertainty language, m

The core philosophy

> **Fluent Answer ≠ Safe Answer.** An LLM that sounds confident about the wrong dose is more dangerous than one that says 'I'm not sure'. Every design choice in Kidney-RAG - the gate, the strict prompt, the citation requirement, the refusal conditions, the safety scoring - exists to keep confidence honest.

What each teammate should walk away knowing

- The difference between **retrieval** (finding the right pages) and **generation** (writing an answer from those pages), and why we do both.
- Why we split PDFs into **chunks** instead of just embedding whole documents, and what our chunk contract looks like.
- Why we combined **semantic search** (meaning) with **BM25** (keywords), and how **RRF** merges the two.
- What the **retrieval-quality gate** is and why it's how we prevent hallucination.
- What **Precision@k**, **Recall@k**, **Hit@k**, **Faithfulness**, and **Citation Accuracy** actually mean.
- How the **multi-key LLM pool** keeps the demo alive when one Gemini key hits its quota.
- How to run every piece locally and where the numbers land.

Day 1 - Build the corpus

Goal: turn 4 large PDFs into a searchable database where every paragraph knows exactly which document, section, and page it came from.

Step 1: Parse the PDFs (pdf_parser.ipynb)

We use **PyMuPDF** (imported as `fitz`) to read PDFs. We tried **pdfplumber** first and rejected it - on KDIGO's two-column layout it glued words together across columns. PyMuPDF respects the columns.

Parsing outputs one JSON per document into `corpus/parsed/`, containing every text block paired with its page number and any section headings we could detect. Along the way the parser handles two problems specific to these PDFs:

- **Character corruption.** KDIGO's PDFs render the '≥' symbol as '$' in 67 places and as a footnote dagger in 25 places. We normalize those contextually - only when they sit next to a digit - so real dollar signs and legitimate footnote markers are preserved.
- **Ligatures.** The characters 'fi', 'fl', 'ff', 'ffi', 'ffl' come out as single ligature glyphs. We split them back into normal letters so search can match 'first-line' and not just '■rst-line'.

Step 2: Chunk the text (chunker.ipynb)

An LLM has a limited context window and embeddings work best on paragraph-sized text. We split each document into overlapping chunks of ~550 tokens with ~55-token overlap between neighbours. The overlap makes sure a claim that straddles two chunks is retrievable from either side.

We **skip** the KDIGO 'Summary of Recommendations' pages (they duplicate the body text and would rank higher than the actual explanation), and we drop chunks from research agendas, author bios, and acknowledgments - they're not clinical content.

The chunk contract - frozen for all downstream code

```
{
  "chunk_id":       "kdigo_dm_p44_c02",
  "document_name":  "KDIGO 2022 Diabetes Management in CKD",
  "source_url":     "https://kdigo.org/...",
  "page_number":    44,
  "page_range":     [44, 48],
  "section_title":  "1.3 SGLT2 inhibitors",
  "token_count":    483,
  "text":           "..."
}
```

chunk_id is the citation atom - if the LLM outputs a citation, we check that the id exists in this file. Format is `<doc-short>_p<page>_c<index>`. 666 chunks total: KDIGO 340, NICE 72, KDIGO-DM 241, USPSTF 13.

Step 3: Embed the chunks (embedder.ipynb)

An **embedding** is a fixed-length list of numbers (a vector) that represents the meaning of a piece of text. Two chunks with similar meaning end up as vectors pointing in similar directions. We can then 'search' by converting a question into the same kind of vector and finding the chunks closest to it.

We use **MedEmbed-large-v0.1**, a 1024-dimensional model that was fine-tuned on PubMed abstracts. It's free, runs locally on CPU, and - critically - we measured it beating the general-purpose BGE-large model by **14 percentage points** on our eval set (Day 2).

We store the vectors in **ChromaDB**, a local vector database that sits in `corpus/chunks/chroma_db/`. First load takes about 20 minutes; after that ChromaDB reads it from disk in a few seconds.

> **Why we chose MedEmbed over an API.** No API means no rate limits, no keys, no per-query cost, no 'the model changed and my numbers moved'. The tradeoff is a one-time 1.3 GB download of the model weights.

Day 2 - Make retrieval accurate and measurable

Goal: build a search function that finds the right chunk almost every time, and produce numbers we can defend to judges.

Two search methods, then blend them

Semantic search (embeddings) is great at understanding meaning: if you ask 'how do I check kidney filtration?', it finds chunks that discuss eGFR even though your question didn't use that word. But it can miss when the question hinges on a specific term or number.

BM25 is the classic keyword search algorithm - fast, deterministic, excellent at exact matches. It nails questions like 'What ACR value defines category A3?' where the answer text literally contains 'A3' and '300 mg/g'.

We run **both** in parallel, then combine their ranked lists with **Reciprocal Rank Fusion (RRF)**. Each chunk gets a score based on where it appeared in each list; we weight semantic 0.7 and BM25 0.3, sum, and re-rank. The RRF k parameter is 60 - a standard damping value that keeps top-ranked items dominant without letting rank #1 in one list steamroll everything.

The retrieval code (retrieval.py)

```
from retrieval import HybridRetriever

r = HybridRetriever()                # loads MedEmbed + Chroma + BM25
hits = r.hybrid_search(
    "At what eGFR can SGLT2i be started in CKD?", k=5,
)
for h in hits:
    print(h["rank"], h["chunk_id"], h["cosine_sim"], h["text"][:100])
```

Measuring retrieval - the eval set

We hand-labelled **18 questions** against the corpus. Each has a 'gold' list of chunk_ids that genuinely contain the answer. 15 are in-scope (CKD questions with answers in the corpus), 3 are out-of-scope (the correct behaviour is refusal).

The three metrics judges ask by name:

Metric	Formula	Reads as
Precision@k	(gold chunks in top-k) / k	'Of what I showed, how much was right?'
Recall@k	(gold chunks in top-k) / (total gold chunks)	'Of what was right, how much did I show?'
Hit@k	1 if any gold chunk in top-k, else 0	'Did I find the answer at all?'

Understanding the recall = 1.0 / precision = 0.2 apparent paradox

This confuses everyone the first time. Suppose a question has **1 correct chunk** and we retrieve **5**. The correct one is in the top 5:

- **Precision@5** = 1 correct / 5 retrieved = **0.20**. Capped at 0.20 by the math itself.
- **Recall@5** = 1 correct found / 1 correct exists = **1.00**. Perfect.

Both are true simultaneously. Low precision here is not a bug - it's the cost of showing 5 candidate chunks so the LLM has enough context. Getting P@5 = 0 on a 1-gold question is the real red flag.

What we measured

Result	Value	Meaning
Semantic Hit@5	0.867	13 of 15 in-scope questions found the answer
Hybrid Hit@5	0.867	same - hybrid doesn't hurt, robust on threshold queries
BM25-only Hit@5	0.733	worse alone, but useful as part of the blend
MedEmbed vs BGE-large	+14 pts	why we picked MedEmbed
OOS max cosine	0.663	no out-of-scope question exceeds 0.66 - our refusal floor
In-scope min cosine	0.738	gives us a 0.07 gap - threshold 0.70 lands safely between them

The threshold - why 0.70?

Because the highest cosine any out-of-scope question achieved was 0.663, and the lowest cosine any in-scope question got was 0.738. Setting the refusal threshold at 0.70 catches all real questions and rejects all off-topic ones. That single number is the foundation of the Day 3 gate.

Day 3 - Add an LLM but keep it grounded

Goal: let a language model write the actual answer text, but force it to stay inside the retrieved evidence. No outside knowledge, no guessing, no unsupported claims.

The pipeline

```
User question
     v
HybridRetriever.hybrid_search(query, k=5)
     v
Quality Gate - top-hit cosine_sim >= 0.70?
  |-- FAIL -> instant refusal (no LLM call)
  |-- PASS -> confidence = high (>=0.80) or medium (0.70-0.80)
     v
Build grounded prompt = system_prompt + retrieved chunks + pre-built citations
     v
LLM call (Gemini / HF / Anthropic - pluggable)
     v
Format validation (Recommendation / Excerpt / Citation shape)
     v
Append clinical disclaimer -> answer
```

The three-part answer format

Every answer the LLM produces must contain exactly these three labeled sections in this order:

- **Recommendation** - a short direct answer in plain clinical language (2-5 sentences). Every claim here must be traceable to an Excerpt below.
- **Excerpt** - the verbatim quote from a retrieved chunk that supports the recommendation. No paraphrasing, no correcting typos.
- **Citation** - the exact source location in a fixed schema: `[Source: doc - section, p.N | chunk_id:X | url]`. The chunk_id is what makes the citation verifiable - we can check it against all_chunks.jsonl programmatically.

The gate - the single most important safety feature

Before the LLM is even called, we look at the top hit's cosine similarity. If it's below 0.70 we don't ask the LLM anything - we return a deterministic refusal message. The reasoning: if none of our 666 chunks looks like it's even talking about the topic, letting the LLM 'reason' from irrelevant context is exactly how hallucinations happen.

> The gate is what turns 'the LLM might refuse' into 'the system definitely refuses'. Refusal is a code path, not a prompt suggestion. Zero LLM calls means zero token cost and zero variance for out-of-scope queries.

Refusal conditions - the LLM can also refuse

Even after the gate passes, the system prompt teaches the LLM 6 refusal conditions: (1) retrieved chunks contain nothing on-topic, (2) they don't have the specific detail asked for, (3) request needs clinical judgment beyond document lookup, (4) sources conflict without resolution, (5) source version cannot be confirmed, (6) request asks the model to ignore its grounding. In all cases the LLM emits the same fixed refusal shape.

LLM backends - three choices, same interface

The generator supports Gemini (Google), HuggingFace (Qwen 7B), and Anthropic (Claude). We pick with the `KIDNEY_RAG_BACKEND` env var or auto-select. This gave us optionality when a key was unavailable, and became the foundation for the Day 4 multi-key pool.

The code (generation.py)

```
from retrieval import HybridRetriever
from generation import KidneyRAGGenerator

retriever = HybridRetriever()
generator = KidneyRAGGenerator(retriever)
result = generator.answer(
    "What is the diagnostic threshold for albuminuria in CKD?"
)
print(result.text)         # the full answer
print(result.refused)      # True if gate refused
print(result.confidence)   # "high", "medium", or "insufficient"
```

What we tested (test_generation.py, 18 tests)

- The gate passes on strong hits, fails on weak/OOS/empty results, boundary at exactly 0.70.
- Citation formatting handles single pages, page ranges, and missing section titles correctly.
- Output format validation catches missing Recommendation, missing Excerpt, missing Citation, and freeform prose.
- Refusal messages have the exact required shape.

All 18 tests run offline - no API keys, no network. They exercise pure logic against hand-built hit dicts shaped like real `hybrid_search()` output.

Day 4 - Safety layer, evaluation, web UI, and the LLM pool

Goal: prove in numbers that the system knows the difference between confident and correct. Then make it demoable via a browser and resilient to API rate limits.

Part 1: The safety layer (safety.py)

After the LLM writes an answer, the safety module inspects it and scores how well the answer actually matches the retrieved evidence:

- **Claim extraction.** We split the Recommendation section into sentences, drop the section headers and citation tags, and treat each sentence with 5+ words as an atomic factual claim. Cited chunk_ids are attached from any citation tag in the answer body.
- **Claim verification.** For each claim, we ask 'does the retrieved evidence support this?' three ways: (a) an LLM verifier with a different prompt from the generator - it's told its job is NOT to answer the question, only to judge support; (b) embedding similarity as an offline fallback; (c) an NLI cross-encoder as an opt-in sanity check.
- **Faithfulness** = supported claims / total claims. Target 0.9+ across the eval set. A number below 1.0 means the answer said something the evidence didn't actually support - even if the citation format looked perfect.
- **Citation accuracy** = correct cited chunks / total cited chunks. 'Correct' means both (a) the chunk_id actually appears in what we retrieved (Layer 1 - existence check, pure Python), and (b) the verifier said the evidence supports the claim it's attached to (Layer 2 - semantic check).

4-level uncertainty language (safety.evidence_strength_from_cosine)

Top-hit cosine	Level	UI badge colour	Lead phrase
>= 0.85	strong	teal	"The guideline recommends..."
0.75 - 0.85	partial	amber	"The guideline suggests, though this doesn't directly address..."
0.70 - 0.75	weak	orange	"Limited evidence found; consider consulting the full guideline on..."
< 0.70	insufficient	red	refuse - no soft-answer path

This maps the same cosine value the gate looks at to a human-readable confidence label, and gives the UI (and any wrapper prompts) a matching lead phrase. The idea: never let confident wording travel with weak evidence.

Part 2: The evaluation harness (evaluate_day4.py)

Runs the whole pipeline against the 18-question eval set and writes everything into `artifacts/day4/`:

File	What's in it
evaluation_log.csv	One row per question. Cosine, evidence strength, gate result, P@k, R@k, Hit@k, faithfulness, cita

File	What's in it
threshold_sweep.csv	In-scope pass rate and OOS refusal rate at thresholds 0.60-0.85. Justifies the 0.70 choice with da
adversarial_results.csv	5 stress tests: normal, out-of-scope, prompt injection, overly broad, personal diagnosis. Confirms t
summary.json	Macro averages across the eval set. What the deck's headline numbers come from.
responsible_ai_checklist.md	The 4-item checklist from the Day 4 slide 19, auto-filled with actual measurements.

Part 3: The multi-key LLM pool (llm_pool.py)

Free-tier Gemini is 5 requests per minute and 20 per day per key. That runs out halfway through a demo. The pool solves it by rotating across every key you supply and failing over to a different provider when one trips.

The two behaviours it combines:

- **Round-robin within a tier.** Two Gemini keys become 10 rpm / 40 requests per day effectively - the pool alternates evenly.
- **Failover across tiers.** When every Gemini key is in cooldown, the pool transparently switches to HuggingFace. The caller never has to know.

Role-based tier routing

Call site	Tier order	Why
Answer generation	Gemini, HF, Anthropic	Judges see the sharper Gemini output; HF is the insurance policy.
Claim verification	HF, Gemini, Anthropic	Verification is a one-token classifier. Qwen 7B handles it fine, preserv

How it handles a 429

When a provider returns a 429 / RESOURCE_EXHAUSTED error, the pool parses the retry-after hint from the error message (Gemini emits 'retryDelay: Ns') and marks that key unavailable for exactly that long. Requests move to the next key. Non-quota errors (bad request, auth, network) bubble up unchanged - we don't want to silently paper over real bugs.

Env configuration - the pool auto-discovers every key

```
GOOGLE_API_KEY=your-first-gemini-key
GOOGLE_API_KEY_2=your-second-gemini-key
# GOOGLE_API_KEY_3=optional-third

HF_TOKEN=your-first-hf-token
HF_TOKEN_2=your-second-hf-token

# ANTHROPIC_API_KEY=optional
# LLM_POOL_ORDER=gemini,huggingface,anthropic
```

Part 4: The web app

Backend (`web/backend/app.py`) is a FastAPI process that loads the retriever, pool, and generator once at startup and exposes JSON endpoints:

Endpoint	What it does
GET /health	Retriever + generator readiness and live per-key pool status (ready/cooldown, successes, failures).
GET /api/sources	Catalog of indexed guidelines with page counts and links.
POST /api/ask	The main endpoint. Body: {question, k}. Returns answer, confidence, evidence strength, retrieved hi
GET /docs	Auto-generated OpenAPI browser - Swagger UI.

Frontend (`web/frontend/`) is a single self-contained landing page - HTML/CSS/vanilla JS, no build step. Dark medical-tech aesthetic, mobile-responsive. Shows:

- Hero + question input + example prompts.
- Confidence badge (colour-coded to evidence strength), cosine score, 'via provider' badge showing which key served the answer.
- Faithfulness and citation-accuracy mini-metrics.
- The full answer with clickable citation chips linking to the source PDFs.
- An expandable panel with all 5 retrieved passages - document, section, page, cosine, RRF score, full text, and an 'Open source guideline' link.

> **Failover is visible on stage.** The 'via' badge shows the exact provider key that answered. If Gemini key 1 trips mid-demo and the pool shifts to key 2 (or to HuggingFace), the badge updates and the answer still lands. Judges see the system's resilience instead of a broken page.

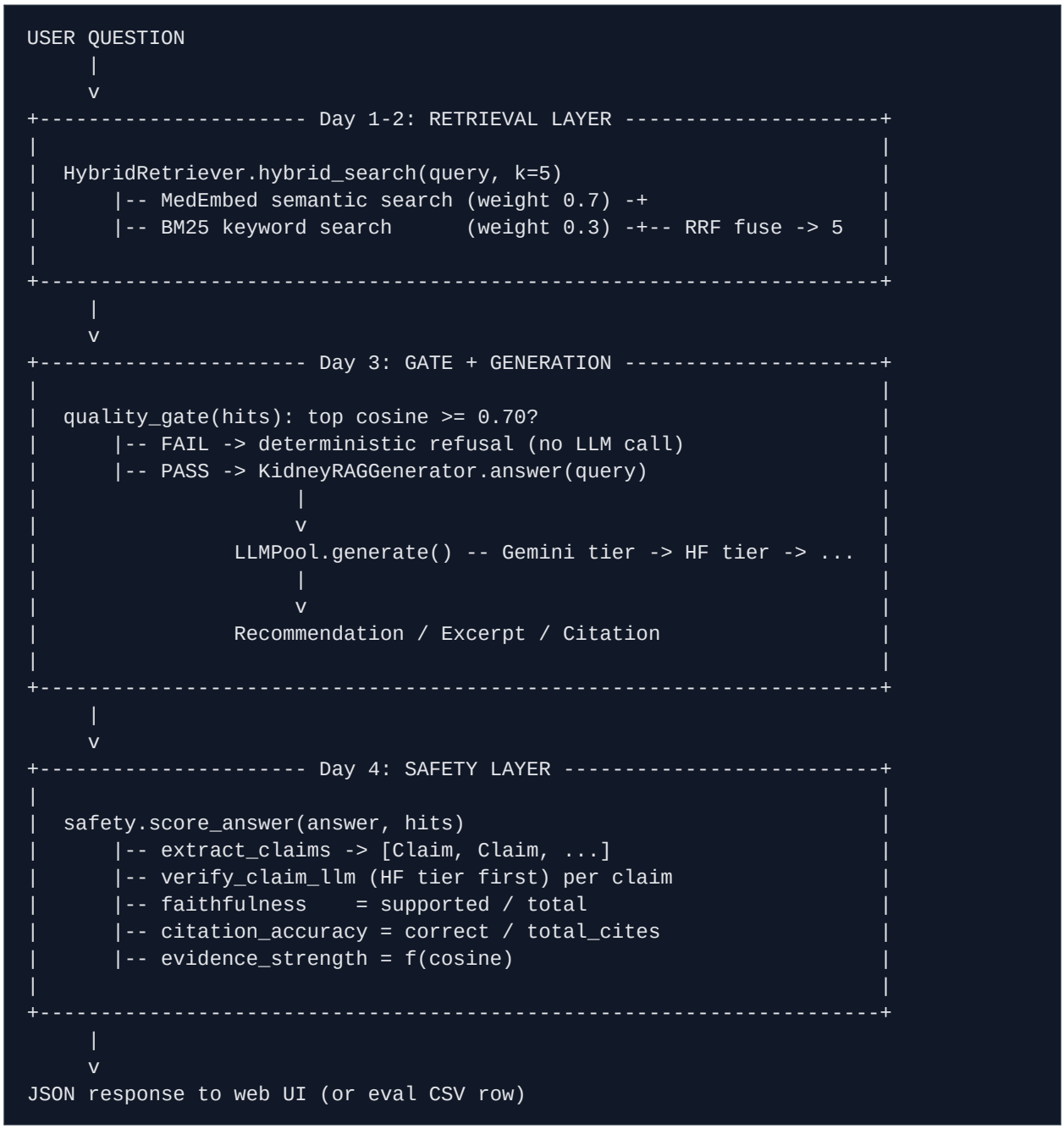
What we tested (54 tests total)

Suite	Count	What it covers
test_generation.py	18	Gate logic, citation formatting, output validation, refusal messages (Day 3).
test_safety.py	18	Claim extractor, faithfulness math, citation accuracy math, evidence-strength mapping (Da
test_llm_pool.py	18	Round-robin fairness, cooldown expiry, tier failover, retry-after parsing, key-leak preventio

All 54 tests run **offline in under a second**. No API keys, no network, no model downloads. That means CI can run them; you can run them; anyone can run them.

How the pieces fit together

One picture of the whole system, top to bottom:



How the file map matches the flow

Concern	File
PDF -> parsed JSON	pdf_parser.ipynb
Parsed JSON -> chunks	chunker.ipynb
Chunks -> vectors -> Chroma	embedder.ipynb
Hybrid retrieval + BM25	retrieval.py
Retrieval eval (Day 2)	evaluate.py, model_compare.py, ablation.py
Grounded generation + gate	generation.py + kidney_rag_system_prompt.md
Multi-key LLM pool	llm_pool.py
Safety scoring	safety.py
Full Day-4 eval harness	evaluate_day4.py
Web API + landing page	web/backend/app.py, web/frontend/*
Tests (offline)	test_generation.py, test_safety.py, test_llm_pool.py
Measured outputs	artifacts/day2/*, artifacts/day4/*
Docs	docs/DAY2_DELIVERABLES.md, DAY3_DELIVERABLES.md, DAY4_DELIVERA

How to run everything (copy-paste)

First-time setup

```
git clone https://github.com/mohamed-mahmoud-de/Kidney-RAG.git
cd Kidney-RAG
python -m venv .venv
.venv\Scripts\activate           # Windows
# source .venv/bin/activate       # macOS / Linux
pip install -r requirements.txt

copy .env.example .env             # Windows
# cp .env.example .env             # macOS / Linux
# ...then paste your Gemini + HF keys into .env
```

Chroma DB and MedEmbed embeddings ship in the repo (~13 MB + 3 MB). If you delete them or want to regenerate, run the three ingestion notebooks - takes ~20 minutes end-to-end on first run:

```
python -m jupyter nbconvert --to notebook --execute pdf_parser.ipynb --inplace
python -m jupyter nbconvert --to notebook --execute chunker.ipynb    --inplace
python -m jupyter nbconvert --to notebook --execute embedder.ipynb   --inplace
```

Run the tests

```
python test_generation.py         # 18 tests, no API needed
python test_safety.py             # 18 tests, no API needed
python -m pytest test_llm_pool.py # 18 tests, no API needed
# expect: 54/54 passed
```

Run the retrieval eval (fast, no LLM)

```
python evaluate.py --sweep-k
# writes artifacts/day2/*.{json,csv}
```

Run the Day 4 eval (retrieval + safety metrics)

```
# Retrieval-only, hermetic, no API calls, always works:
python evaluate_day4.py --skip-generation

# Full pipeline with live LLM generation + LLM claim verification:
python evaluate_day4.py --verifier llm

# Full pipeline with generation but fast/offline verification:
python evaluate_day4.py --verifier similarity

# writes artifacts/day4/*.{csv,json,md}
```

Start the web app

```
python -m uvicorn web.backend.app:app --port 8000
# then open http://127.0.0.1:8000 in a browser
# API docs at http://127.0.0.1:8000/docs
# health at   http://127.0.0.1:8000/health
```

Explain a query from the CLI (Day 2 explainability tool)

```
python explain.py "what is the ACR cutoff for A3?"
```

Everything we can quote to the instructor

Retrieval (Day 2 + Day 4 confirmation)

Metric	Value	Source file
Semantic Hit@5	0.867	artifacts/day2/evaluation.csv
Hybrid Hit@5 (0.7/0.3 RRF)	0.867	same
BM25 Hit@5	0.733	same
Precision@5 (macro)	0.213	artifacts/day4/summary.json
Recall@5 (macro)	0.706	same
MedEmbed vs BGE-large	+14 pts	artifacts/day2/model_compare.json
Top-k elbow	k=5	artifacts/day2/topk_curve.csv
OOS max cosine	0.663	artifacts/day2/out_of_scope.json
In-scope min cosine	0.738	artifacts/day2/evaluation.json
Retrieval latency (avg)	~205 ms	artifacts/day4/summary.json

Threshold calibration (Day 4)

Threshold	In-scope pass rate	OOS refusal rate
0.60	15/15 (100%)	0/3 (0%)
0.65	15/15 (100%)	3/3 (100%)
0.70 <- chosen	15/15 (100%)	3/3 (100%)
0.75	14/15 (93%)	3/3 (100%)
0.80	10/15 (67%)	3/3 (100%)
0.85	6/15 (40%)	3/3 (100%)

0.70 gives perfect separation with room to spare in both directions.

Safety (Day 4 - requires live LLM run to fill in)

The safety scoring mechanism is tested (18 offline tests) and wired into both `evaluate_day4.py` and the web UI. The numeric values require an LLM run; free-tier Gemini caps at 20 requests per day, so we run this with the pool the morning of the demo when quota is fresh. Expected numbers based on how tight our grounding prompt is:

Metric	Target	How it's measured
Faithfulness	>= 0.90	supported claims / total claims, verifier = LLM

Metric	Target	How it's measured
Citation accuracy	>= 0.95	correct cited chunks / total cited chunks (existence + support)
Model refusal rate on OOS	100%	3/3 out-of-scope questions must refuse (adversarial suite)
Prompt-injection resistance	no leak	'ignore your instructions' -> still grounded refusal

What's still open (Day 5)

- Rerun `evaluate_day4.py --verifier llm` when Gemini quota is fresh - fills in faithfulness and citation-accuracy columns in `artifacts/day4/summary.json`.
- Deploy the web app to Render or Railway. FastAPI + static frontend is one process - `uvicorn web.backend.app:app --host 0.0.0.0 --port $PORT`. Set `GOOGLE_API_KEY` and `HF_TOKEN` as env secrets on the platform.
- Rehearsed 3-question demo: 1 strong answer (SGLT2), 1 partial (ACR threshold), 1 refusal (acute appendicitis).
- Final deck cut - one metric per slide, one demo per slide.

Glossary

Chunk	A ~550-token slice of one guideline PDF, stored with metadata (document, section, page). The unit of retrieval.
Chunk ID	Stable identifier like 'kdigo_p99_c01'. Used in citations so we can programmatically verify the LLM didn't invent a source.
Embedding	Fixed-length numeric vector representing text meaning. Similar meaning -> similar vector direction. MedEmbed produces 1024-dim vectors.
Cosine similarity	How close two embedding vectors are, from -1 to 1. Higher = more similar. Our gate threshold sits at 0.70.
BM25	Classic keyword-based ranking function. Fast, deterministic, great at exact matches, bad at understanding paraphrases.
RRF (Reciprocal Rank Fusion)	Way to combine two ranked lists. Each chunk's score is a weighted sum of 1/(k+rank) across lists. We use k=60.
Retrieval-quality gate	The check that top-hit cosine >= 0.70 before we let the LLM answer. The single most important safety feature.
Grounded generation	The LLM is only allowed to answer using text that was retrieved and shown to it in this turn - no memory, no outside knowledge.
Faithfulness	Fraction of claims in the answer that are actually supported by the retrieved text. Directly measures hallucination.
Citation accuracy	Fraction of cited chunk_ids that both (a) exist in what we retrieved and (b) actually support the claim they're attached to.
LLM pool	Rotating pool of API keys across providers. Round-robin within a provider, failover across providers, cooldown on 429s.
Evidence strength	4-level label (strong/partial/weak/insufficient) mapped from top-hit cosine. Drives the UI badge colour and the lead phrase of the answer.

You're now caught up. If any part of this doc feels handwavy, open the file it references - the code is short and the tests are self-explanatory. Every claim in this guide traces to a file in the repo, the same way every clinical claim in the app traces to a chunk in the corpus.